

SMART LENDER

AI-Powered Loan Approval Prediction System

Project Description

Smart Lender is an AI-powered loan approval prediction system developed to assist financial institutions in evaluating loan applications efficiently and accurately. The application uses Machine Learning techniques to analyze applicant details such as gender, marital status, dependents, education, employment status, applicant income, co-applicant income, loan amount, loan term, credit history, and property area to predict whether a loan is likely to be **approved** or **rejected**.

The project is built using **Python, Flask, Scikit-learn, Pandas, NumPy, HTML**, and **CSS**. A **Random Forest Classifier** is trained on a historical loan dataset to learn patterns associated with loan approval decisions. The trained model is then integrated into a Flask web application, allowing users to enter applicant information through a simple web interface and receive an instant prediction along with a confidence score.

The primary objective of Smart Lender is to reduce manual effort in the loan evaluation process, improve the speed and consistency of decision-making, and demonstrate the practical application of Machine Learning in the banking and financial sector. With a model accuracy of **83.74%**, the application provides reliable predictions and serves as an effective decision-support tool for loan eligibility assessment.

Scenarios

Scenario 1: Loan Approval Prediction

A user enters valid applicant details such as income, education, employment status, credit history, loan amount, and property area. The system analyzes these inputs using the trained Random Forest model and predicts whether the loan is likely to be approved or rejected.

Scenario 2: Loan Eligibility Assessment

The application evaluates the applicant's financial profile and credit history to determine loan eligibility. This helps financial institutions make quick and data-driven lending decisions.

Scenario 3: High-Risk Applicant Detection

When an applicant has a poor credit history, low income, or other unfavorable factors, the system identifies the application as high risk and predicts loan rejection, helping reduce potential financial losses.

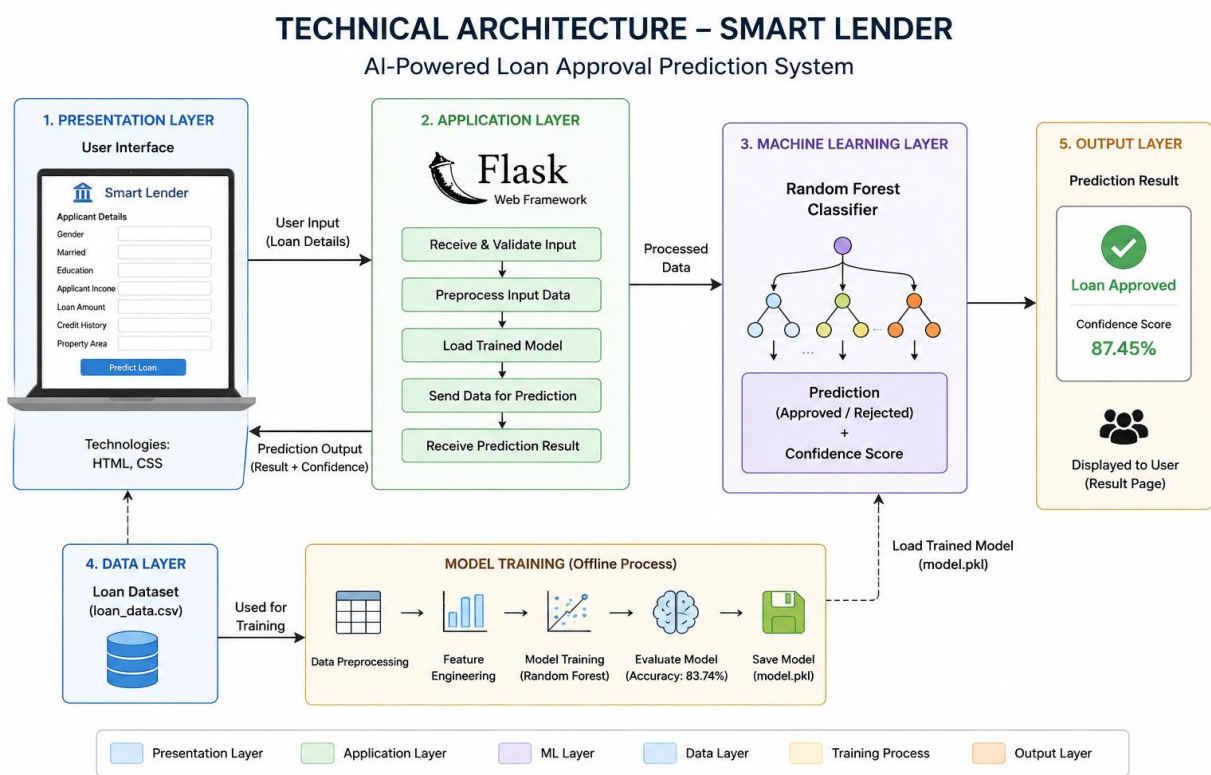
Scenario 4: User-Friendly Web-Based Prediction

Users interact with a simple web interface to submit loan application details. The system processes the information instantly and displays the prediction result along with the confidence score, making the application easy to use.

Scenario 5: Decision Support for Financial Institutions

The Smart Lender system acts as a decision-support tool for banks and financial organizations by providing fast, consistent, and accurate loan approval predictions based on historical data patterns.

Technical Architecture



Description: The Smart Lender system follows a layered architecture to provide accurate loan approval predictions. Users enter loan applicant details through the web interface developed using HTML and CSS. The Flask backend receives the input, validates it, and performs the necessary preprocessing before passing the data to the trained Random Forest machine learning model (model.pkl). The model analyzes the applicant's information and predicts whether the loan should be approved or rejected, along with a confidence score. Finally, the prediction result is displayed to the user through the result page. The loan dataset is used during the model training process to build and evaluate the prediction model, ensuring reliable loan eligibility assessment.

Pre-requisites:

Requirement	Purpose
Windows 10/11	Operating System
Python 3.11	Programming Language
Visual Studio Code	Code Editor
Flask	Web Application Framework
Pandas	Data Processing
NumPy	Numerical Computations
Scikit-learn	Machine Learning Model Development
Joblib	Saving and Loading the Trained Model
HTML & CSS	User Interface Development
Loan Prediction Dataset	Model Training and Testing
Web Browser	Running the Application

Required Python Libraries

- Flask
- Pandas
- NumPy
- Scikit-learn
- Joblib

Project Workflow

The Smart Lender project was developed using a structured workflow consisting of five milestones. Each milestone focuses on a specific stage of the project, from setting up the development environment to deploying the application. This approach ensured systematic development, proper testing, and successful implementation of the AI-powered loan prediction system.

Milestone 1: Development Environment Setup

This milestone focuses on preparing the development environment required for building the Smart Lender application.

Activity 1.1: Setting Up the Development Environment

- Install Python 3.11 and Visual Studio Code.
- Create and activate a Python virtual environment.
- Install required Python libraries using requirements.txt.
- Configure the project folder structure.
- Download and organize the loan prediction dataset.

Activity 1.2: Project Initialization

- Create the Flask application files (app.py, train_model.py).

- Create frontend folders (templates and static).
 - Create the README and requirements files.
 - Verify the development environment by running the Flask application.
-

Milestone 2: Machine Learning Model Development

This milestone focuses on building and training the loan prediction model.

Activity 2.1: Data Preparation

- Load the loan prediction dataset.
- Handle missing values.
- Perform data preprocessing and cleaning.
- Encode categorical variables into numerical values.

Activity 2.2: Model Training

- Split the dataset into training and testing sets.
 - Train the Random Forest Classifier.
 - Evaluate model performance.
 - Save the trained model as model.pkl.
-

Milestone 3: Backend Development

This milestone focuses on implementing the Flask backend and integrating the trained model.

Activity 3.1: Flask Application Development

- Configure Flask routes.
- Load the trained machine learning model.
- Receive and validate user inputs.
- Process prediction requests.

Activity 3.2: Prediction Processing

- Convert user inputs into model-compatible format.
 - Generate loan approval predictions.
 - Calculate the confidence score.
 - Send prediction results to the frontend.
-

Milestone 4: Frontend Development

This milestone focuses on designing an interactive and user-friendly interface.

Activity 4.1: User Interface Design

- Develop the loan application form using HTML.
- Design responsive pages using CSS.
- Create a clean and simple user interface.

Activity 4.2: Result Display

- Display loan approval or rejection status.
 - Show prediction confidence score.
 - Enable users to perform multiple predictions.
-

Milestone 5: Deployment and Testing

This milestone focuses on verifying the application and preparing it for deployment.

Activity 5.1: Local Deployment

- Run the Flask application locally.
- Verify all project functionalities.
- Test multiple loan prediction scenarios.

Activity 5.2: Documentation and Project Submission

- Prepare project documentation.
 - Create README.md and requirements.txt.
 - Upload the project to GitHub.
 - Prepare the project for SmartBridge submission.
-

Milestone 6: Project Conclusion

This milestone summarizes the successful completion of the Smart Lender project and highlights the overall outcomes achieved during development.

Milestone 1: Development Environment Setup

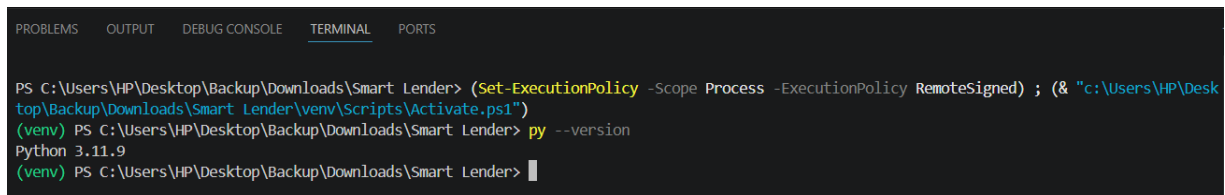
Objective

The objective of this milestone is to set up the complete development environment required for the Smart Lender application. This includes installing the necessary software, configuring Python, creating a virtual environment, installing dependencies, organizing the project structure, and preparing the dataset for machine learning model development.

Activity 1.1: Install Python and Visual Studio Code

Step 1: Install Python

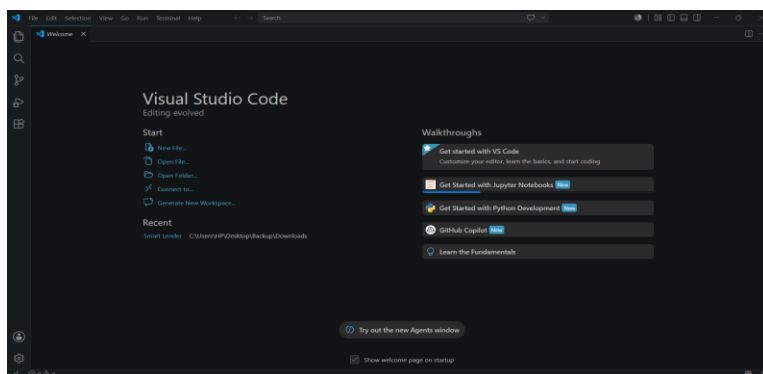
- Download and install **Python 3.11** from the official Python website.
- During installation, enable the "**Add Python to PATH**" option.
- Verify the installation using: `py --version`



```
PS C:\Users\HP\Desktop\Backup\Downloads\Smart Lender> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "c:\Users\HP\Desktop\Backup\Downloads\Smart Lender\venv\Scripts\Activate.ps1")
(venv) PS C:\Users\HP\Desktop\Backup\Downloads\Smart Lender> py --version
Python 3.11.9
(venv) PS C:\Users\HP\Desktop\Backup\Downloads\Smart Lender> |
```

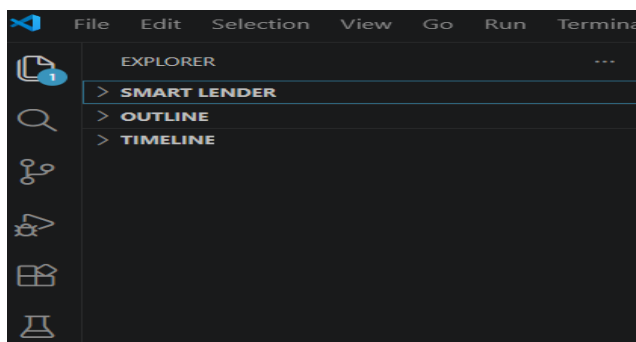
Step 2: Install Visual Studio Code

- Download and install **Visual Studio Code** from the official website.
- Launch VS Code after installation.



Activity 1.2: Create Project Folder

- Create a project folder named: **Smart Lender**.
- Open this folder in Visual Studio Code.



Activity 1.3: Create Virtual Environment

- Open the VS Code terminal and execute: `py -m venv venv`
- This creates a virtual environment named **venv** to isolate project dependencies.

Activity 1.4: Activate Virtual Environment

- In PowerShell execute: `.\venv\Scripts\Activate`

- If PowerShell blocks script execution, temporarily allow it using: `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`
- Then activate again. Successful activation displays: (venv) before the terminal prompt.

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\HP\Desktop\Backup\Downloads\Smart Lender> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "c:\Users\HP\Desktop\Backup\Downloads\Smart Lender\venv\Scripts\Activate.ps1")
(venv) PS C:\Users\HP\Desktop\Backup\Downloads\Smart Lender>

```

Activity 1.5: Install Required Libraries

- Install all required libraries using: `pip install flask pandas numpy scikit-learn joblib`
- Or install directly from the requirements file: `pip install -r requirements.txt`

```

(venv) PS C:\Users\HP\Desktop\Backup\Downloads\Smart Lender> pip list
Package Version
-----
anyio 4.14.1
argon2-cffi 25.1.0
argon2-cffi-bindings 25.1.0
arrow 1.4.0
asttokens 3.0.1
async-lru 2.3.0
attrs 26.1.0
babel 2.18.0
beautifulsoup4 4.15.0
bleach 6.4.0
blinker 1.9.0
certifi 2026.6.17
cffi 2.0.0
charset-normalizer 3.4.7
click 8.4.2
colorama 0.4.6
comm 0.2.3

```

Activity 1.6: Organize Project Structure

Create the following folders:

- dataset/
- templates/
- static/
- screenshots/
- documents/

Create the following files:

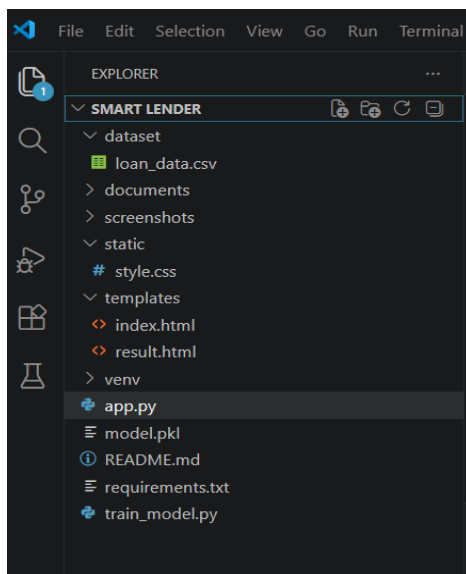
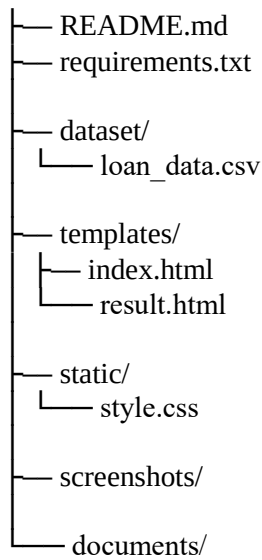
- app.py
- train_model.py
- README.md
- requirements.txt

Final project structure:

```

Smart Lender/
├── app.py
├── train_model.py
└── model.pkl

```

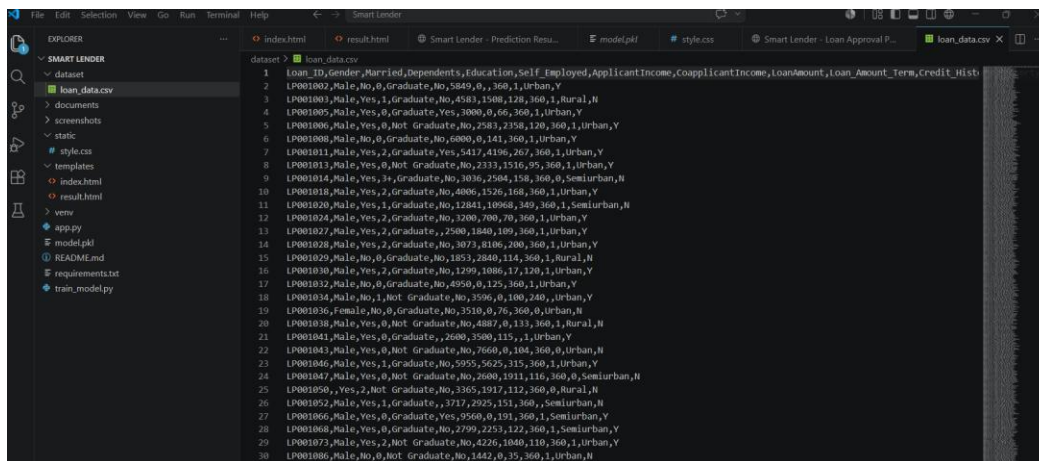


Activity 1.7: Download the Dataset

Download the **Loan Prediction Dataset** and place the CSV file inside the **dataset** folder.

The dataset contains:

- Gender
- Married
- Dependents
- Education
- Self Employed
- Applicant Income
- Coapplicant Income
- Loan Amount
- Loan Amount Term
- Credit History
- Property Area
- Loan Status



Activity 1.8: Verify Flask Application

- Run the application using: `py app.py`
- If the terminal displays: Running on <http://127.0.0.1:5000> . open the browser and visit: <http://127.0.0.1:5000>
- The Smart Lender home page should load successfully.

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\HP\Desktop\Backup\Downloads\Smart Lender> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "c:\Users\HP\Desktop\Backup\Downloads\Smart Lender\venv\Scripts\Activate.ps1")
(venv) PS C:\Users\HP\Desktop\Backup\Downloads\Smart Lender> py train_model.py
Model Accuracy: 83.74%
model.pkl saved successfully.
(venv) PS C:\Users\HP\Desktop\Backup\Downloads\Smart Lender>
* History restored

PS C:\Users\HP\Desktop\Backup\Downloads\Smart Lender> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "c:\Users\HP\Desktop\Backup\Downloads\Smart Lender\venv\Scripts\Activate.ps1")
(venv) PS C:\Users\HP\Desktop\Backup\Downloads\Smart Lender> py app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 696-465-987

```

The image shows the web application interface for 'Smart Lender'. It features a header with the 'Smart Lender' logo and the title 'Loan Approval Prediction System'. Below the header, there are several input fields for user information: 'Gender' (dropdown menu with 'Male' selected), 'Married' (dropdown menu with 'Yes' selected), 'Dependents' (text input field with '0' entered), 'Education' (dropdown menu with 'Graduate' selected), 'Self Employed' (dropdown menu with 'Yes' selected), 'Applicant Income' (text input field), and 'Coapplicant Income' (text input field). The interface is clean and modern, with a light blue and white color scheme.

Milestone 2: Machine Learning Model Development

Objective

The objective of this milestone is to develop a machine learning model capable of predicting loan approval status based on applicant information. This involves loading the dataset, preprocessing the data, training the Random Forest Classifier, evaluating the model's performance, and saving the trained model for integration with the Flask web application.

Activity 2.1: Load the Dataset

Step 1: Import Required Libraries

Import all the required Python libraries for data processing and machine learning.

```
train_model.py > ...
1  import pandas as pd
2  import joblib
3
4  from sklearn.model_selection import train_test_split
5  from sklearn.metrics import accuracy_score
6  from sklearn.ensemble import RandomForestClassifier
7
```

Step 2: Load the Dataset

Load the loan prediction dataset using Pandas.

```
8  # Load dataset
9  data = pd.read_csv("dataset/loan_data.csv")
```

Activity 2.2: Data Preprocessing

Step 3: Handle Missing Values

Replace missing values with appropriate values such as the mode or median.

```
11 # Fill missing values
12 data["Gender"] = data["Gender"].fillna(data["Gender"].mode()[0])
13 data["Married"] = data["Married"].fillna(data["Married"].mode()[0])
14 data["Dependents"] = data["Dependents"].fillna(data["Dependents"].mode()[0])
15 data["Self_Employed"] = data["Self_Employed"].fillna(data["Self_Employed"].mode()[0])
16 data["LoanAmount"] = data["LoanAmount"].fillna(data["LoanAmount"].median())
17 data["Loan_Amount_Term"] = data["Loan_Amount_Term"].fillna(data["Loan_Amount_Term"].mode()[0])
18 data["Credit_History"] = data["Credit_History"].fillna(data["Credit_History"].mode()[0])
19
```

Step 4: Encode Categorical Data

Convert categorical values into numerical values.

Example:

- Male → 1
- Female → 0
- Yes → 1
- No → 0

```

20 # Convert categorical values
21 data["Dependents"] = data["Dependents"].replace("3+", 3)
22 data["Dependents"] = pd.to_numeric(data["Dependents"])
23
24 data["Gender"] = data["Gender"].map({"Male":1,"Female":0})
25 data["Married"] = data["Married"].map({"Yes":1,"No":0})
26 data["Education"] = data["Education"].map({"Graduate":1,"Not Graduate":0})
27 data["Self_Employed"] = data["Self_Employed"].map({"Yes":1,"No":0})
28 data["Property_Area"] = data["Property_Area"].map({
29     "Rural":0,
30     "Semiurban":1,
31     "Urban":2
32 })
33 data["Loan_Status"] = data["Loan_Status"].map({"N":0,"Y":1})
34

```

Activity 2.3: Train the Machine Learning Model

Step 5: Split Dataset

Split the dataset into training and testing datasets.

```

42 # Split Dataset
43 X_train, X_test, y_train, y_test = train_test_split(
44     X,
45     y,
46     test_size=0.20,
47     random_state=42,
48     stratify=y
49 )

```

Step 6: Train Random Forest Classifier

Train the machine learning model.

```

51 # Train Random Forest
52 model = RandomForestClassifier(
53     n_estimators=300,
54     max_depth=12,
55     min_samples_split=4,
56     random_state=42
57 )
58
59 model.fit(X_train, y_train)

```

Activity 2.4: Evaluate the Model

Step 7: Predict Test Data

- Generate predictions using the testing dataset.
- Calculate model accuracy.

```

(venv) PS C:\Users\HP\Desktop\Backup\Downloads\Smart Lender> py train_model.py
Model Accuracy: 83.74%
model.pkl saved successfully.

```

Activity 2.5: Save the Trained Model

Step 8: Save the Model

Save the trained model for future predictions.

```
67 # Save Model
68 joblib.dump(model, "model.pkl")
69
70 print("model.pkl saved successfully.")
```

Milestone 3: Backend Development

Objective

The objective of this milestone is to develop the backend of the Smart Lender application using the Flask framework. This includes integrating the trained Random Forest model with the web application, processing user inputs, generating loan predictions, and displaying the results through the web interface.

Activity 3.1: Develop the Flask Application

Step 1: Import Required Libraries

Import the necessary libraries for building the Flask application and loading the trained machine learning model.

```
app.py > ...
1 from flask import Flask, render_template, request
2 import joblib
3 import pandas as pd
```

Step 2: Initialize the Flask Application

- Create the Flask application and load the trained model.
- This initializes the Flask server and loads the trained Random Forest model for prediction.

```
5 app = Flask(__name__)
5
7 model = joblib.load("model.pkl")
```

Activity 3.2: Create Application Routes

Step 3: Create the Home Route

- Define the home route to display the loan application form.
- This route loads the **Home Page** when users access the application.

```
9 @app.route("/")
10 def home():
11     return render_template("index.html")
```

Step 4: Create the Prediction Route

- Create a route to receive applicant details and generate predictions.
- This route accepts user input submitted from the loan application form.

```

12
13 @app.route("/predict", methods=["POST"])
14 def predict():

```

Activity 3.3: Process User Input

Step 5: Read Applicant Details

- Retrieve the applicant details entered by the user.
- Similarly, read all remaining applicant details required for prediction.
- The input values are then converted into a Pandas DataFrame before passing them to the machine learning model.

```

6 input_data = pd.DataFrame([
7     "Gender": int(request.form["gender"]),
8     "Married": int(request.form["married"]),
9     "Dependents": int(request.form["dependents"]),
10    "Education": int(request.form["education"]),
11    "Self_Employed": int(request.form["self_employed"]),
12    "ApplicantIncome": float(request.form["applicant_income"]),
13    "CoapplicantIncome": float(request.form["coapplicant_income"]),
14    "LoanAmount": float(request.form["loan_amount"]),
15    "Loan_Amount_Term": float(request.form["loan_amount_term"]),
16    "Credit_History": float(request.form["credit_history"]),
17    "Property_Area": int(request.form["property_area"])
18 ])

```

Activity 3.4: Generate Loan Prediction

Step 6: Predict Loan Status

- Generate the prediction using the trained Random Forest model.
- Calculate the confidence score.

```

30 prediction = model.predict(input_data)[0]
31
32 confidence = model.predict_proba(input_data)[0]
33 confidence_score = max(confidence) * 100
34
35 if prediction == 1:
36     result = "✅ Loan Approved"
37 else:
38     result = "❌ Loan Rejected"

```

Activity 3.5: Display Prediction Result

Step 7: Display the Result

Return the prediction to the result page.

The result page displays:

- Loan Status
- Confidence Score

```

40 return render_template(
41     "result.html",
42     prediction=result,
43     confidence=f"{confidence_score:.2f}%"
44 )

```

Activity 3.6: Run and Verify the Application

Step 8: Run the Flask Server

- Execute: `py app.py`
- The terminal displays: `* Running on http://127.0.0.1:5000`

```
(venv) PS C:\Users\HP\Desktop\Backup\Downloads\Smart Lender> py app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 696-465-987
```

The image displays the Smart Lender Loan Approval Prediction System interface. On the left is a form for inputting applicant details. To the right are two panels showing the results of loan predictions.

Smart Lender Loan Approval Prediction System Input Form:

- Gender: Female
- Married: Yes
- Dependents: 1
- Education: Graduate
- Self Employed: Yes
- Applicant Income: 50000
- Coapplicant Income: 10000

Loan Prediction Results:

- Result 1:** Loan Approved, Confidence Score: 50.53%
- Result 2:** Loan Rejected, Confidence Score: 69.45%

Milestone 4: Frontend Development

Objective

The objective of this milestone is to design and develop an interactive, user-friendly, and responsive web interface for the Smart Lender application. This includes creating the loan application form, designing the result page, applying CSS styling, and ensuring smooth interaction between the frontend and the Flask backend.

Activity 4.1: Design the Home Page

Step 1: Create the Home Page (index.html)

Develop the main user interface where users can enter loan applicant details. The form includes fields such as gender, marital status, dependents, education, employment status, applicant income, co-applicant income, loan amount, loan term, credit history, and property area.

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Smart Lender - Loan Approval Prediction</title>
7   <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
8 </head>
9 <body>
10
11 <div class="container">
12   <h1>Smart Lender</h1>
13   <h3>Loan Approval Prediction System</h3>
14
15   <form action="/predict" method="POST">
16     <label>Gender</label>
17     <select name="gender" required>
18       <option value="1">Male</option>
19       <option value="0">Female</option>
20     </select>
21
22     <label>Married</label>
23     <select name="married" required>
24       <option value="1">Yes</option>
25       <option value="0">No</option>
26     </select>
27
28     <label>Dependents</label>
29     <select name="dependents" required>
30       <option value="0">0</option>
31       <option value="1">1</option>
32       <option value="2">2</option>
33       <option value="3">3+</option>
34     </select>
35
36     <label>Education</label>
37

```

Step 2: Create the Loan Application Form

Design an HTML form that collects all required applicant information and submits it to the Flask backend through the /predict route.

The form includes:

- Gender
- Married
- Dependents
- Education
- Self Employed
- Applicant Income
- Co-applicant Income
- Loan Amount
- Loan Amount Term
- Credit History
- Property Area

Smart Lender
Loan Approval Prediction System

Gender
Female

Married
Yes

Dependents
1

Education
Graduate

Self Employed
Yes

Applicant Income
50000

Coapplicant Income
10000

Loan Amount
300000

Loan Amount Term (Months)
24

Credit History
Good (1)

Property Area
Urban

Predict Loan

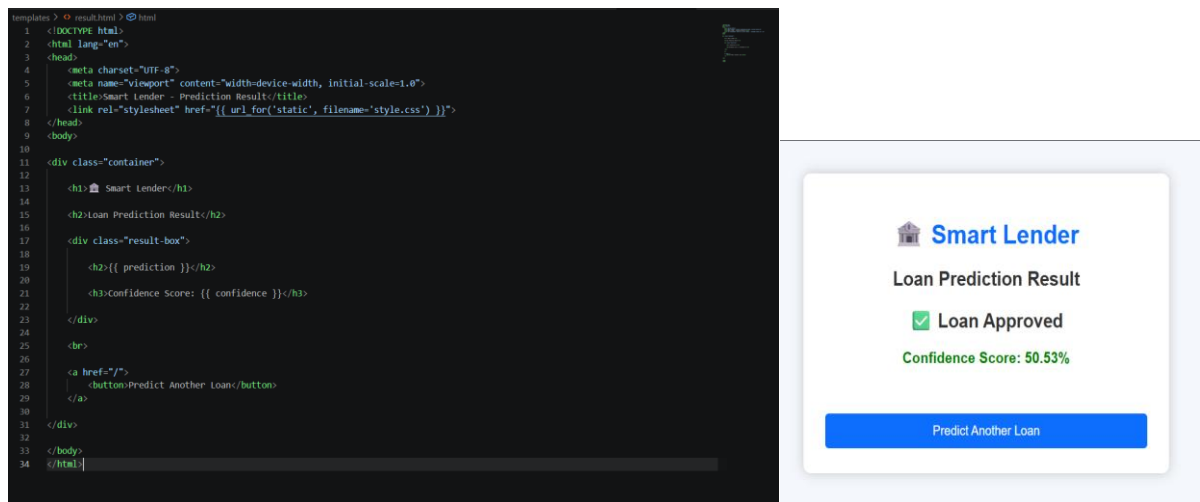
Activity 4.2: Design the Result Page

Step 3: Create the Result Page (result.html)

Develop the result page to display the loan prediction outcome after processing the user's input.

The page displays:

- Loan Status (Approved/Rejected)
- Confidence Score
- Button to perform another prediction



Activity 4.3: Apply CSS Styling

Step 4: Create the Stylesheet (style.css)

Develop a CSS file to improve the appearance and responsiveness of the application.

The stylesheet includes:

- Background styling
- Typography
- Form layout
- Buttons
- Input fields
- Result card
- Responsive design


```

static > # style.css > buttonhover
1
2 body{
3   margin:0;
4   padding:0;
5   background-color:#f4f7fc;
6   font-family:Arial, Helvetica, sans-serif;
7 }
8
9 .container{
10  width:450px;
11  margin:40px auto;
12  background-color:white;
13  padding:30px;
14  border-radius:10px;
15  box-shadow:0px 0px 15px rgba(0,0,0,0.2);
16 }
17
18 h1{
19   text-align:center;
20   color:#0d6efd;
21 }
22
23 h2{
24   text-align:center;
25   color:#333;
26 }
27
28 h3{
29   text-align:center;
30   color:green;
31 }
32
33 label{
34   display:block;
35   margin-top:12px;
36   font-weight:bold;
37 }

```

Step 5: Enhance User Experience

Improve the interface by:

- Creating a clean and professional layout.
- Highlighting prediction results.
- Displaying confidence scores clearly.
- Making the interface easy to navigate.

Activity 4.4: Validate User Inputs

Step 6: Perform Form Validation

- Ensure that all mandatory fields are completed before the form is submitted.
- The application validates user input and prevents submission when required fields are missing.

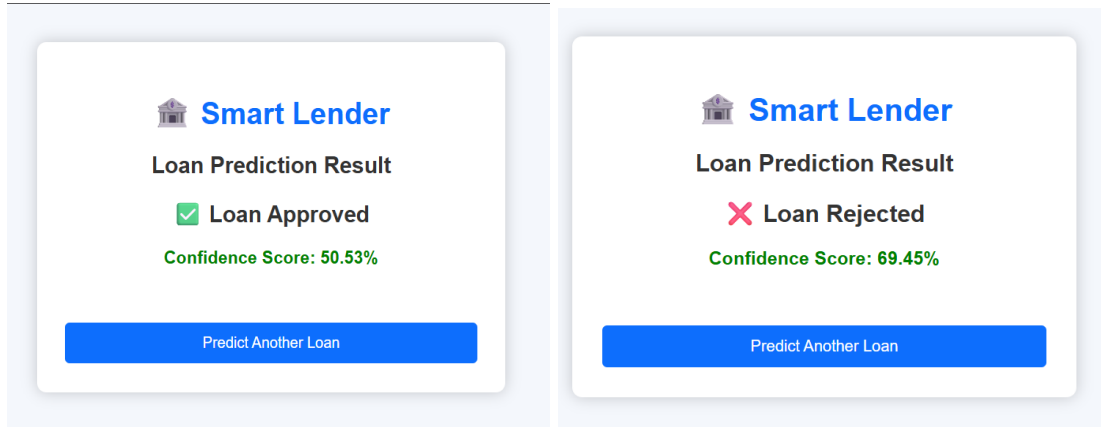
The screenshot shows a web form for loan prediction. It includes several dropdown menus and text input fields. A validation error is shown for the 'Loan Amount' field, which is currently empty. The error message is 'Please fill out this field.' displayed in a small box next to the field. The form fields are: 'Graduate' (dropdown, value: Graduate), 'Self Employed' (dropdown, value: Yes), 'Applicant Income' (text input, value: 30000), 'Coapplicant Income' (text input, value: |), 'Loan Amount' (text input, value: 200000, with error message), 'Loan Amount Term (Months)' (text input, value: 12), 'Credit History' (dropdown, value: Good (1)), and 'Property Area' (dropdown, value: Semiurban). A blue 'Predict Loan' button is at the bottom.

Activity 4.5: Verify Frontend Functionality

Step 7: Test User Interface

Verify that:

- The Home Page loads correctly.
- The form accepts valid inputs.
- Prediction results are displayed successfully.
- The "Predict Again" button redirects to the Home Page.
- The interface is responsive and user-friendly.



Milestone 5: Project Deployment

Objective

The objective of this milestone is to deploy the Smart Lender application, verify its functionality in the deployment environment, and make the project available for demonstration and future development. The application was successfully deployed in the local environment using the Flask development server, and the project files were organized for version control using GitHub.

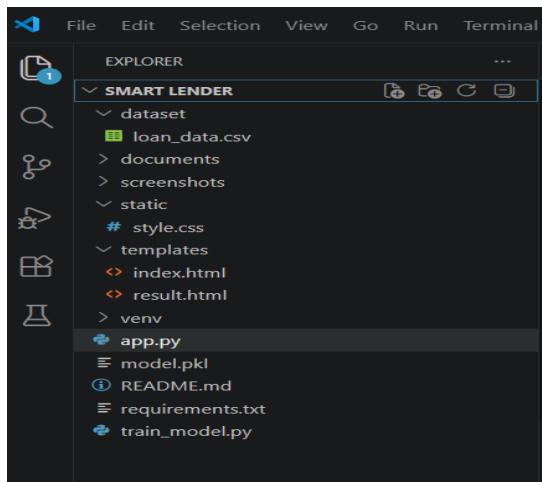
Activity 5.1: Prepare the Project for Deployment

Step 1: Organize Project Files

Ensure that all required project files are available and properly organized before deployment.

The project includes:

- app.py
- train_model.py
- model.pkl
- requirements.txt
- README.md
- dataset/loan_data.csv
- templates/
- static/



Step 2: Verify Project Dependencies

- Confirm that all required Python libraries are installed using the `requirements.txt` file.
- Run the following command: `pip install -r requirements.txt`
- This installs all dependencies required to execute the application successfully.

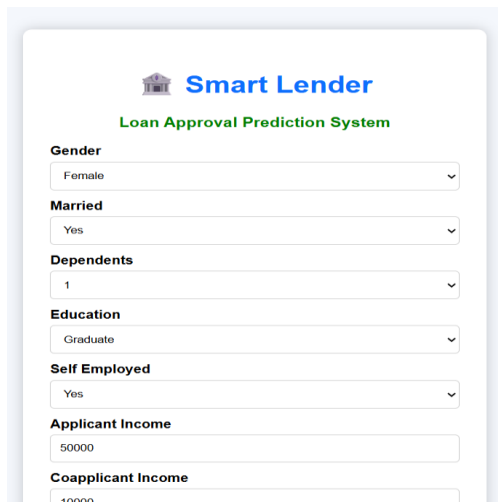
```
requirements.txt
1  Flask==3.1.2
2  pandas==2.3.1
3  numpy==2.3.1
4  scikit-learn==1.7.1
5  joblib==1.5.1
```

Activity 5.2: Deploy the Application Locally

Step 3: Launch the Flask Application

- Start the application using the Flask development server.
- Successful execution displays: * Running on `http://127.0.0.1:5000`
- Open the application in a web browser using: `http://127.0.0.1:5000`

```
(venv) PS C:\Users\HP\Desktop\Backup\Downloads\Smart Lender> py app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 696-465-987
```



Smart Lender
Loan Approval Prediction System

Gender
Female

Married
Yes

Dependents
1

Education
Graduate

Self Employed
Yes

Applicant Income
50000

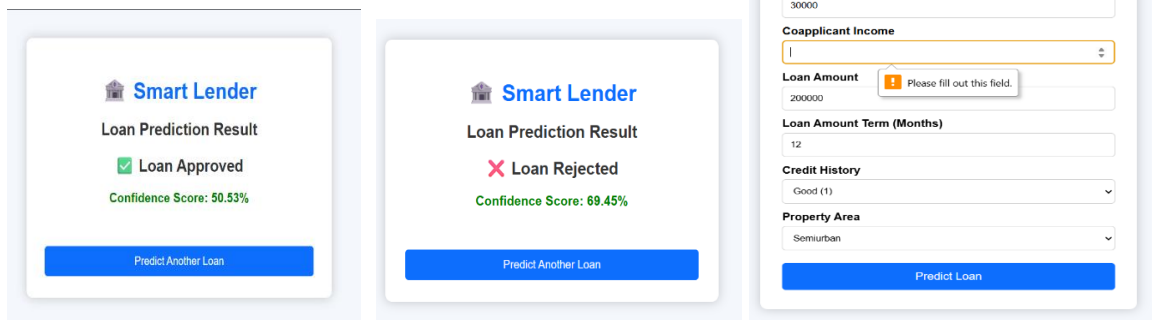
Coapplicant Income
10000

Step 4: Verify Application Functionality

Test the application using different loan applicant details.

Verify that:

- Loan Approved predictions are displayed correctly.
- Loan Rejected predictions are displayed correctly.
- Confidence score is displayed.
- Form validation works properly.



The first two screenshots show the 'Loan Prediction Result' page. The left one displays 'Loan Approved' with a green checkmark and a confidence score of 50.53%. The right one displays 'Loan Rejected' with a red X and a confidence score of 69.45%. Both have a 'Predict Another Loan' button.

The third screenshot shows the input form with the following details: Education: Graduate, Self Employed: Yes, Applicant Income: 30000, Coapplicant Income: 1, Loan Amount: 200000 (with a validation error message 'Please fill out this field.'), Loan Amount Term (Months): 12, Credit History: Good (1), Property Area: Semiurban. It has a 'Predict Loan' button.

Activity 5.3: Version Control Using GitHub

Step 5: Upload the Project to GitHub

Create a GitHub repository named **Smart Lender** and upload all project files.

The repository includes:

- Source Code
- Machine Learning Model
- Dataset
- README.md
- requirements.txt

- Documentation

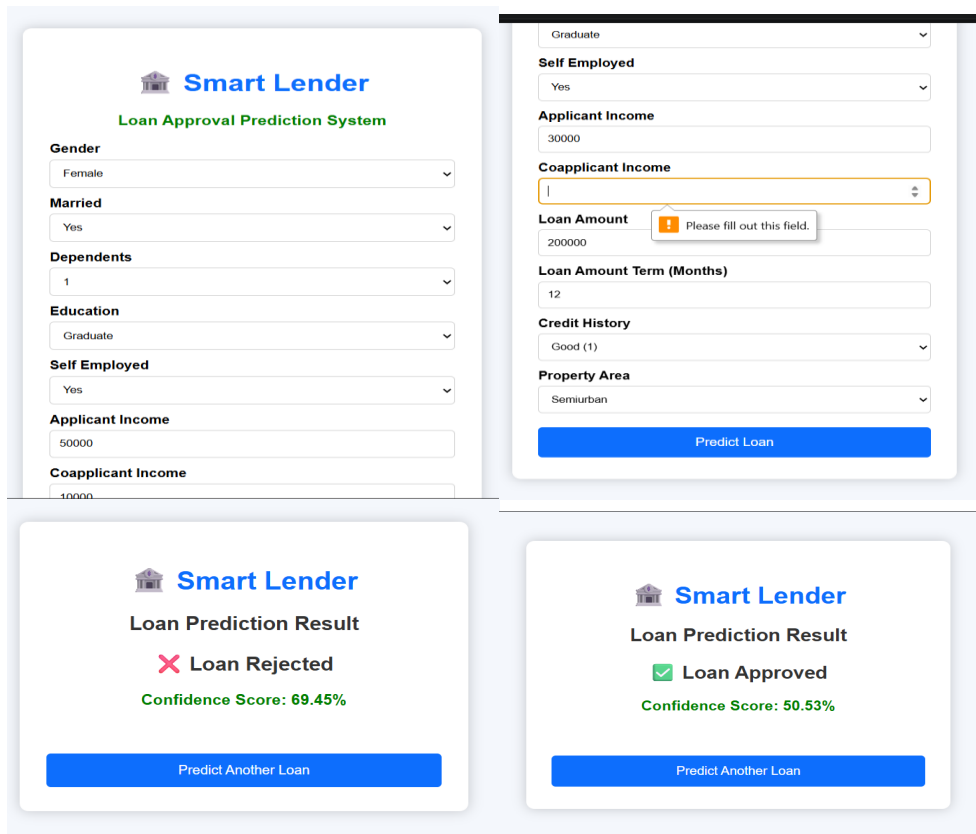
The GitHub repository enables version control, collaboration, and easy project sharing.

Activity 5.4: Final Verification

Step 6: Validate the Complete Application

Perform a final check to ensure:

- The application starts successfully.
- All pages load correctly.
- Predictions are accurate.
- The confidence score is displayed.
- The user interface is responsive.
- No runtime errors occur during execution.



The image displays four screenshots of the 'Smart Lender' web application. The top-left screenshot shows the 'Loan Approval Prediction System' form with fields for Gender (Female), Married (Yes), Dependents (1), Education (Graduate), Self Employed (Yes), Applicant Income (50000), and Coapplicant Income (10000). The top-right screenshot shows a detailed view of the form with a warning message 'Please fill out this field.' for the Loan Amount field. The bottom-left screenshot shows the 'Loan Prediction Result' as 'Loan Rejected' with a 'Confidence Score: 69.45%'. The bottom-right screenshot shows the 'Loan Prediction Result' as 'Loan Approved' with a 'Confidence Score: 50.53%'. All screenshots feature a 'Predict Another Loan' button at the bottom.

Milestone 6: Conclusion

The **Smart Lender – AI-Powered Loan Approval Prediction System** was successfully designed and developed using **Python, Flask, and Machine Learning**. The application predicts loan approval status based on applicant information and provides quick, accurate, and reliable results through a user-friendly web interface. The Random Forest model achieved an accuracy of **83.74%**, making the system effective for loan eligibility prediction. This project demonstrates the practical application of machine learning in the banking and financial sector while improving the efficiency and consistency

of the loan approval process. With future enhancements such as database integration, cloud deployment, and real-time banking APIs, the Smart Lender application can be further improved for real-world use.